ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC KINH TẾ

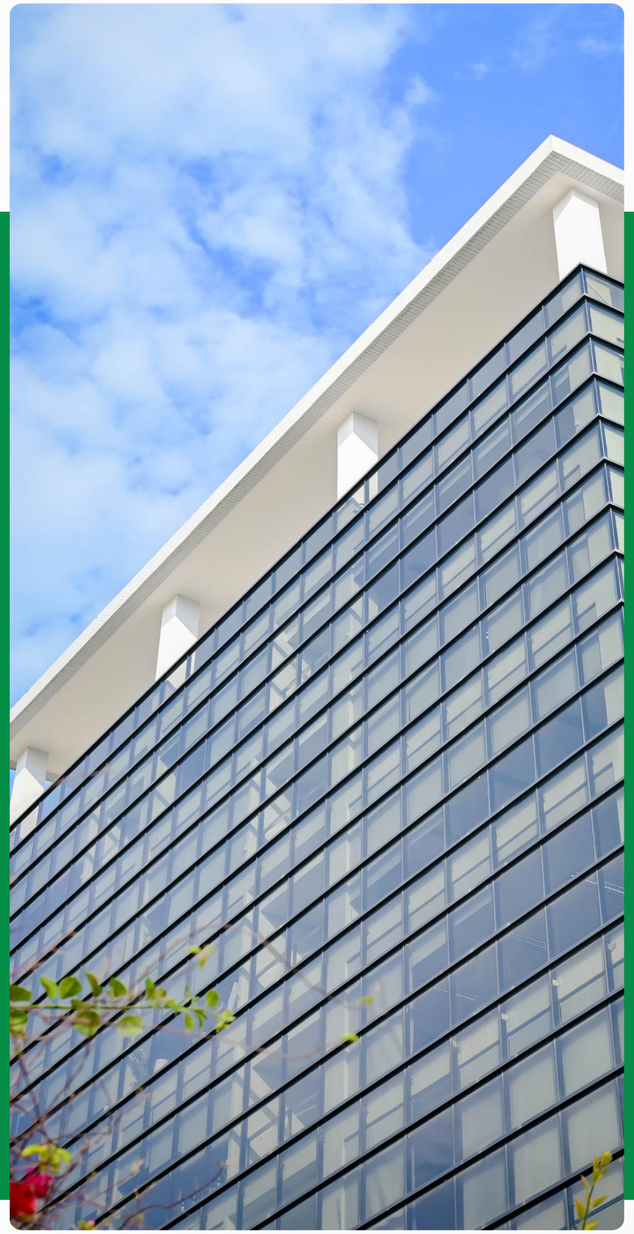
# CHƯƠNG 6

# QUẢN LÝ TẬP TIN




**LẬP TRÌNH WEB**

Khoa Thống kê – Tin học

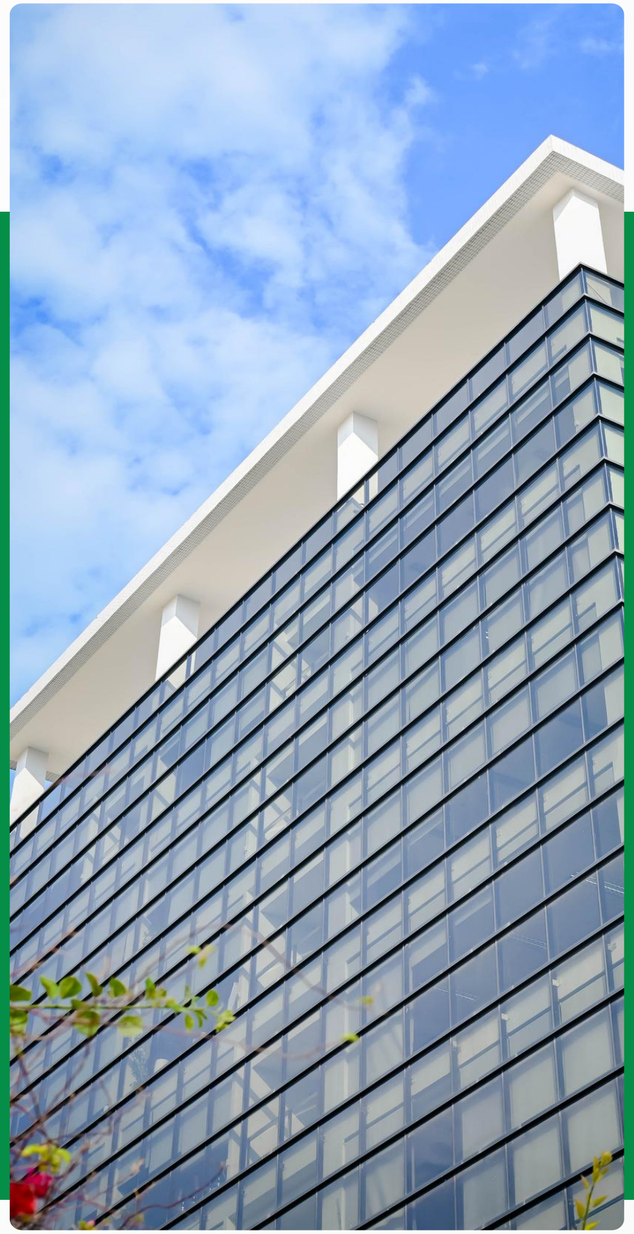


# MỤC TIÊU CHƯƠNG

LO1 - Giải thích được các tính năng của Django phục vụ tìm kiếm và truy xuất các tập tin tĩnh

LO2 - Giải thích được cách thức tải lên và lưu trữ các tập tin trên máy chủ web

LO3 - Giải thích được các thư viện của Django phục vụ tải lên, lưu trữ và truy xuất các tập tin trên máy chủ web



# HỌC LIỆU

TL2 - Shaw, B., Badhwar, S., Guest, C., & KS, B. C. (2023), *Web Development with Django: A definitive guide to building modern Python web applications using Django 4*, Packt Publishing Ltd.

# 6.1 Giới thiệu chung



LO1

# Tập tin tĩnh và Tập tin tài liêu

## Tập tin tĩnh

- Nằm trên thư mục thuộc website
- Là một phần của ứng dụng ban đầu (vd: dùng trong giao diện,...)
- Gồm mã HTML, CSS và Javascript, hình ảnh
- Có thể quản lý thông qua app staticfiles có sẵn của Django

## Tập tin tài liệu

- Do người dùng tải lên hoặc ứng dụng sinh ra trong quá trình vận hành
- Gồm dữ liệu đa phương tiện (hình ảnh, video, audio) và tài liệu (Word, Excel, PDF, text,...)
- Django cung cấp các thư viện để hỗ trợ tải lên, quản lý và trình bày theo dạng phù hợp

# 6.2 Quản lý tập tin tĩnh



# Hàm static

## Hàm static

Chỉ sử dụng trong giai đoạn phát triển, với các thiết đặt:

- Trong tập tin **settings.py**:
  - **DEBUG = True**
  - **INSTALLED_APPS** có chứa app **django.contrib.staticfiles**
- Trong tập tin **urls.py**
  - **urlpatterns** có chứa ánh xạ:

    **path(settings.STATIC_URL, django.conf.urls.static)**

  → Bất cứ URL nào có đường dẫn bắt đầu bằng **settings.STATIC_URL** (mặc định là **/static/**) sẽ ánh xạ đến hàm **static** (trong **View**)

# Các công cụ tìm kiếm tập tin tĩnh

- Hàm **static** (từ View): chuyển đường dẫn trong URL thành đường dẫn đến vị trí tập tin trên đĩa

  ***Ví dụ:*** *URL có đường dẫn* ***/static/logo.png*** *→ đường dẫn đến tập tin trên đĩa là* ***bookr/static/logo.png***
- Lệnh **collectstatic**: xác định và thu tập toàn bộ các tập tin trong thư mục dự án rồi chép sang một thư mục duy nhất để triển khai ứng dụng web
- Lệnh **findstatic**: nhận vào tên tập tin tính và trả về đường dẫn đầy đủ của tập tin đó

# Các lớp hỗ trợ tìm kiếm tập tin

Được thiết đặt ở tham số **STATICFILES_FINDERS**, mặc định là 2 lớp:

- **Lớp AppDirectoriesFinder** (django.contrib.staticfiles.finders.AppDirectoriesFinder)
  - Dùng để tìm kiếm các tập tin tĩnh trong thư mục con static của mỗi App
  - Tổ chức tập tin tĩnh theo App giúp hạn chế các vấn đề khi truy xuất các tập tin có tên giống nhau trong các App khác nhau
    - Lưu ý: App cần được đăng ký trong biến INSTALLED_APPS của tập tin settings.py
- **Lớp FileSystemFinder** (django.contrib.staticfiles.finders.FileSystemFinder)
  - Dùng để tìm kiếm các tập tin tĩnh nằm trên toàn hệ thống trong các thư mục được xác định bởi biến STATICFILES_DIRS (trong tập tin settings.py)
  - STATICFILES_DIRS có thể là:
    - Danh sách các thư mục cụ thể
    - Danh sách các bộ ở dạng (tiền tố, thư mục)

# Sinh URL với thẻ template static

- Việc sử dụng đường dẫn tuyệt đối đến tập tin tĩnh trong mã nguồn ứng dụng web gây ra nhiều khó khăn khi cần cập nhật, điều chỉnh vị trí đặt tập tin
- App **staticfiles** cung cấp thẻ static để sinh URL động cho các tập tin trong template
  - Thẻ này cũng có thể hỗ trợ kiểm soát lưu bộ đệm (cache) trên trình duyệt
- Cần tải app này vào template với thẻ `{% load static %}`
- Cú pháp: `{% static 'đường_dẫn' %}`

  ***Ví dụ:*** `{% static 'landing/logo.png' %}` → `/static/landing/logo.png`
- Có thể sử dụng "as [tên_biến]" để gán kết quả xuất ra thành 1 biến và dung ở các vị trí khác trên template

  ***Ví dụ:*** `<img src="{% static 'logo.png' as logo_path %}">`

  *Sau đó có thể sử dụng biến* ***logo_path*** *ở vị trí khác:* `<img src="{{ logo_path }}">`

# Cung cấp phiên bản tập tin mới nhất

- **Lưu đệm (caching):** tăng tốc độ thực thi/mở trang web bằng cách lưu lại kết quả và dùng lại ở các lần truy xuất sau

  → Có thể gặp vấn đề (cache invalidation) khi các tập tin lưu đệm quá hạn (đã có bản mới): Việc giới hạn thời gian lưu cache chưa khắc phục được triệt để
- Phương án của Django: Thêm **mã băm (hash)** vào tên tập tin khi dùng lệnh **collectstatic** để triển khai ứng dụng:
  - Chỉ dùng được khi dùng bộ lưu trữ **ManifestFilesStorage**
  - Sinh ra một tập tin tên là **staticfiles.json** để ánh xạ tên các tập tin gốc và tên tập tin có mã băm
  - Các phiên bản cũ hơn của tập tin (kèm mã băm cũ) vẫn sẽ được lưu lại để truy xuất khi cần thiết

  **Lưu ý:** Có thể định nghĩa các bộ lưu trữ tùy chỉnh (Xem trang 249-251 trong Giáo trình)

# 6.3 Tải tập tin lên máy chủ



LO2

# Thiết đặt thư mục chứa dữ liệu tải lên

- Sử dụng 2 tham số thiết đặt trong tập tin **settings.py** (tương tự STATIC_ROOT và STATIC_URL):
  - **MEDIA_ROOT**: Đường dẫn trên đĩa đến thư mục gốc lưu dữ liệu đa phương tiện
  - **MEDIA_URL**: Là URL được dùng để trỏ đến thư mục lưu dữ liệu đa phương tiện

  ***Lưu ý:*** *Vì lý do bảo mật, thư mục MEDIA_ROOT phải không trùng với STATIC_ROOT*
- Tương tự các tập tin tĩnh, MEDIA_ROOT phải là thư mục riêng và có thể truy xuất trực tiếp để tránh việc Django phải xử lý các yêu cầu truy xuất tập tin (ảnh hưởng hiệu năng hệ thống)
- Trong giai đoạn phát triển, Django có thể xử lý các yêu cầu truy xuất nhưng cần phải khai báo ánh xạ URL:

```
from django.conf.urls.static import static
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

# Thiết đặt truy xuất tập tin tải lên

- Để sử dụng MEDIA_URL trong template, cần truyền tham số này trong biến ngữ cảnh của hàm render()

  ***Ví dụ:***

```
from django.conf import settings
def my_view(request):
    return render(request, "template.html", {"MEDIA_URL":settings.MEDIA_URL,
"username": "jbloggs"})
```

  - Việc này cũng có thể thực hiện tự động với một bộ xử lý ngữ cảnh (context processor) – Hàm với 1 tham số là biến request:

```
from django.conf import settings
def media(request):
    return {"MEDIA_URL": settings.MEDIA_URL}
```

- Sử dụng MEDIA_URL trong template như một biến: **{{ MEDIA_URL }}**

  ***Ví dụ:*** `<img src="{{ MEDIA_URL }}uploads/image.jpg">`

# Thiết kế form chọn tập tin tải lên

## Sử dụng HTML

- Thiết đặt 2 tham số method và enctype trong thẻ <form>

```
<form method="post" enctype="multipart/form-data">
```

- Sử dụng thẻ <input> với type là file để tạo hộp chọn tập tin tải lên:

```
<input type="file" name="file-upload-name">
```

## Tạo hộp chọn file tải lên sử dụng Django forms

- Sử dụng lớp **`forms.FileField`** với các thuộc tính:
  - **max_length**: chiều dài tối đa của tên tập tin
  - **allow_empty_file**: Cho phép tải lên những tập tin rỗng (kích thước bằng 0)

# Xử lý tập tin tải lên

- Các tập tin tải lên được lưu trong **request.FILES** hoặc **cleaned_data** (của form)
- Đọc nội dung tải lên: dùng phương thức **read()**

  ***Ví dụ:*** `content = request.FILES["file-upload-name"].read()`
- Lưu tập tin: dùng phương thức **write()**

  ***Lưu ý:*** *Tập tên tải lên có kích thước dưới 2.5MB sẽ được đặt trong bộ nhớ, nếu tập tin lớn hơn sẽ được lưu vào thư mục tạm trên đĩa.*

→ *Cần lưu tập tin vào thư mục chỉ định để truy xuất lại về sau*

- Để tránh tập tin quá lớn ảnh hưởng đến hiệu năng hệ thống, có thể đọc theo từng đoạn với phương thức **chunks()**

  ***Ví dụ:***

```
with open("/path/to/output.jpg", "wb+") as output_file:
    uploaded_file = request.FILES["file-upload-name"]
        for chunk in uploaded_file.chunks():
            output_file.write(chunk)
```

# Xử lý tập tin tải lên

- Có thể định nghĩa một hàm **save_file_upload** để lưu tập tin tải lên như sau:

```
def save_file_upload(uploaded_file, save_path):
    with open(save_path, "wb+") as output_file:
        for chunk in uploaded_file.chunks():
            output_file.write(chunk)
```

- Một số thuộc tính của trường tập tin tải lên:
  - **size**: Kính thước của tập tin tải lên
  - **name**: Tên của tập tin tải lên
  - **content_type**: Loại tập tin tải lên, như hình ảnh (image/jpeg), văn bản (text/plain),...
  - **charset**: Bộ mã hóa ký tự (dùng cho tập tin văn bản)

  ***Lưu ý:*** *Giá trị của các thuộc tính name , content_type và charset được cung cấp từ trình duyệt web → Có thể phát sinh các vấn đề về bảo mật. Sinh viên đọc thêm ở mục Security and trust of browsers' sent values (trang **379-381** của **Giáo trình**)*

# Tải tập tin hình ảnh lên máy chủ

- Hộp chọn tập tin hình ảnh để tải lên: **forms.ImageField**
  - Chức năng tương tự **FileField** nhưng tự động kiểm tra tính hợp lệ của dữ liệu tập tin (đúng là tập tin hình ảnh)
  - Có thêm thuộc tính **image**: là một đối tượng **PIL Image** dùng để kiểm tra tính hợp lệ
    - Thư viện **Pillow (PIL)** cung cấp nhiều chức năng để thao tác với hình ảnh

  ***Lưu ý:*** *Sau khi kiểm tra tính hợp lệ của tập tin xong thì đối tượng **PIL Image** sẽ được đóng lại (để tiết kiệm bộ nhớ và tránh mở quá nhiều tập tin)*

  *→ Chỉ đọc được một số thông số, không lấy được nội dung hình ảnh (nếu không đọc lại tập tin)*

  ***Ví dụ:***
  - *Khai báo form:*

```
class ExampleForm(forms.Form):
    picture = forms.ImageField()
```

  - *Truy xuất thông số hình ảnh*

```
image = form.cleaned_data["picture"].image
w = image.width # Chiều rộng, số pixel
h = image.height # Chiều cao, số pixel
f = image.format # Định dạng tập tin, như: "PNG"
```

# Tải tập tin hình ảnh lên máy chủ

- Truy xuất dữ liệu hình ảnh: dùng phương thức **image.getdata()**

  ***Lưu ý:*** *Cần đọc tập tin trước khi truy xuất dữ liệu hình ảnh*

```
from PIL import Image
image = Image.open(picture_field)
image.getdata()
```

- Thay đổi kích thước hình ảnh: dùng phương thức **image.thumbnail()**

```
size = 50, 50
image.thumbnail(size)
```

Lưu ý: Sau khi thay đổi kích thước thì cần lưu lại tập tin

```
image.save("path/to/file.png",                    "JPEG")
```

# 6.4 Quản lý tập tin dữ liệu với Django



LO3

# Kiểm soát truy cập vào tập tin

Các máy chủ web thường không cung cấp khả năng kiểm soát truy cập theo người dùng ứng dụng web

Cần có cơ chế quản lý riêng cho các tập tin cần bảo vệ (giới hạn truy cập)

- Cách 1: Tạo một thư mục con chứa các tập tin cần bảo vệ và phân quyền chỉ cho Django truy xuất
- Cách 2: Tạo một view riêng để truy xuất tập tin từ đĩa
  - Kiểm tra người dùng có quyền truy cập đến tập tin không
  - Trả về nội dung tập tin tương ứng với đường dẫn tương đối sử dụng lớp FileRespone
  - Lớp FileRespone có thể phân đoạn và gửi tập tin đến máy khách để giảm thiểu tài nguyên cần dùng

# Kiểm soát truy cập vào tập tin

***Ví dụ:*** *Một view kiểm tra và chỉ cho phép người dùng đã đăng nhập được tải tập tin:*

```
import os.path
from django.conf import settings
from django.http import FileResponse
from django.core.exceptions import PermissionDenied
def download_view(request, relative_path):
    if request.user.is_anonymous:
        raise PermissionDenied
    full_path = os.path.join(settings.MEDIA_ROOT,"protected", relative_path)
    file_handle = open(full_path, "rb")
    return FileResponse(file_handle)
```

*Có thể tạo một ánh xạ URL đến view này, dạng như:*

```
urlpatterns = [
    …
    path("downloads/<path:relative_path>", views.download_view)
]
```

# Lưu trữ tập tin với đối tượng Model

- Các tùy chọn lưu trữ tập tin của tham số **upload_to** của trường **FileField**:
  - Đường dẫn đến thư mục cụ thể

    ***Ví dụ:*** *Lưu trữ các tập tin trong thư mục files bên trong MEDIA_ROOT*

    ```
    file_field = models.FileField(upload_to="files/")
    ```

    Hoặc: `file_field = models.FileField(upload_to=pathlib.Path("files/"))`
  - Tổ chức lưu trữ theo ngày tháng tải lên:

    ***Ví dụ:*** *Lưu trữ các tập tin theo phân cấp thư mục dạng Năm/Tháng/Ngày trong MEDIA_ROOT/files*

    ```
    file_field = models.FileField(upload_to="files/%Y/%m/%d/")
    ```
  - Sử dụng một hàm xử lý:

    ***Ví dụ:*** *Lưu trữ các tập tin theo tài khoản người dùng, có thể định nghĩa 1 hàm dạng:*

    ```
    def user_grouped_file_path(instance, filename):
        return "{}/{}/{}/{}".format(instance.username,filename[0].lower(),
    filename[1].lower(), filename)
    ```

    *Sử dụng hàm trong tham số upload_to:*

    ```
    file_field = models.FileField(upload_to=user_grouped_file_path)
    ```

# Lưu trữ tập tin với đối tượng Model

- Thực hiện lưu trữ các tập tin tải lên từ trường FileField: dùng phương thức save()

***Ví dụ:***

```
class ExampleForm(forms.Form):
    uploaded_file = forms.FileField()

def view(request, model_pk):
    form = ExampleForm(request.POST, request.FILES)
    if form.is_valid():
        m = ExampleModel.object.get(pk=model_pk)
        m.file_field = form.cleaned_data["uploaded_file"]
        m.save()
    return render(request, "template.html")
```

***Lưu ý:***

- Có thể thực hiện việc lưu trữ hình ảnh với trường ImageFile theo cách tương tự với FileField
- Sinh viên có thể tham khảo thêm các tùy chọn nâng cao trong làm việc với tập tin trong phần **Working with FieldFile** trang 403-408 của Giáo trình.

# Lưu trữ tập tin với ModelForm

- Cần truyền tham số request.FILES khi khởi tạo đối tượng ModelForm

***Ví dụ:***

```
def view(request):
    if request.method == "POST":
        form = ExampleModelForm(request.POST, request.FILES)
        form.save()
        return redirect("/success-page")
    else:
        form = ExampleModelForm()
    return (request, "template.html", {"form": form}
```

Lưu ý: Trường hợp sử dụng phương thức lưu không commit (form.save(False)) thì dữ liệu sẽ không được lưu lại và cần phải gọi hàm save() một lần nữa để lưu dữ liệu và tập tin

***Ví dụ:*** *Tiếp tục từ ví dụ trên*

```
        form = ExampleModelForm(request.POST,request.FILES)
        m = form.save(False)
        m.attribute = "value" #Gán giá trị tùy ý cho thuộc tính rồi lưu lại
        m.save()
```

# Truy xuất đến tập tin

- URL của tập tin cần truy xuất có thể lấy từ thuộc tính url của trường FileField

  ***Ví dụ:***

  - *Truy xuất tập tin trong view:*

```
instance = ExampleModel.objects.first()
url = instance.file_field.url # Get the URL
```

  - *Truy xuất tập tin trong template:*

```
<img src="{{ instance.file_field.url }}">
```

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC KINH TẾ

# HẾT CHƯƠNG

KHOA THỐNG KÊ – TIN HỌC

